PRÁCTICA 7

Semáforo de salida: mide tu tiempo de reacción

DOCUMENTO	ASIGNATURA	TITULACIÓN	CENTRO
Del alumnado	**Arq. de Computadores II**	**Grado en Ing. Informática · 2.º**	**ESEI · Universidade de Vigo**

Contenido

1. Ficha de la práctica 3

2. De qué va 3

3. Cómo es una partida 4

3.1 Lo que se ve en el display 5

4. El problema interesante: ¿de dónde sale el azar? 5

5. Medir bien el tiempo de reacción 7

6. Desarrollo paso a paso 8

Paso 1. La secuencia de luces 8

Paso 2. El apagón y la medida 8

Paso 3. La espera impredecible 8

Paso 4. La salida falsa 8

Paso 5. El récord y el torneo 8

Paso 6. Comprueba dónde se sella el instante 9

7. Criterios de aceptación 9

8. Criterios de evaluación 10

9. Entregables 11

10. Uso de asistentes de inteligencia artificial 11

Anexo A. Ampliaciones, para quien acabe pronto 13

Anexo B. Conexionado 13

1. Ficha de la práctica

CAMPO	VALOR
Sesión	Sesión 11 de 13 · práctica final
Duración	2 horas presenciales
Modalidad	Individual, en el laboratorio
Requisitos previos	P2, P3, P4 y P5. Esta práctica las usa todas
Material	Placa NUCLEO-F401RE · módulo LED&KEY (TM1638) · STM32CubeIDE · biblioteca TM1638
Entregables	Proyecto CubeIDE, tabla de tiempos y memoria breve (ver apartado 9)
Plazo	Una semana desde el final de la sesión, por Moovi

2. De qué va

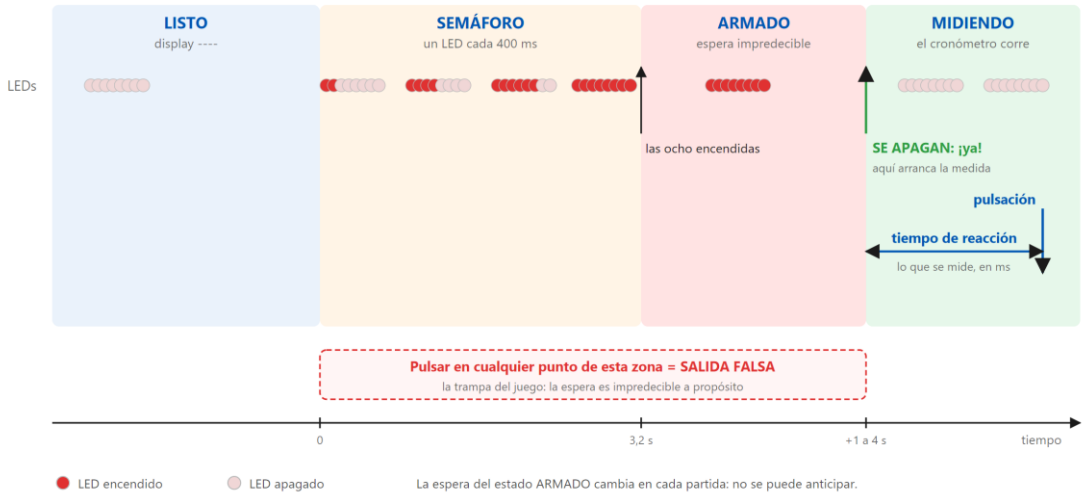
En la parrilla de un Gran Premio se encienden ocho luces rojas, una a una. Cuando llevan todas encendidas un rato **que nadie sabe cuánto va a durar**, se apagan de golpe. Ese apagón es la señal de salida, y lo que separa a un piloto bueno de uno normal son un par de centésimas.

Vas a construir exactamente eso con los ocho LEDs del módulo, y a medir tu propio tiempo de reacción en milisegundos. La máquina guarda el mejor tiempo de la sesión.

> Es la práctica final y no introduce ningún concepto nuevo: **usa los cuatro anteriores a la vez**. Máquina de estados de P2, medida de tiempo sin deriva de P3, interrupción

> externa de P4, y una pregunta que viene de P5: de dónde se saca azar en un chip que no tiene generador de números aleatorios.

3. Cómo es una partida



LAS CUATRO FASES DE UNA PARTIDA Y LO QUE SE MIDE.

1. **LISTO.** Display en `----`, LEDs apagados. Una pulsación arranca la partida.
2. **SEMÁFORO.** Los LEDs se encienden uno a uno, de izquierda a derecha, cada 400 ms. En total 3,2 segundos.
3. **ARMADO.** Las ocho encendidas. Aquí la máquina espera **entre 1 y 4 segundos**, y esa duración cambia en cada partida.
4. **MIDIENDO.** Los LEDs se apagan de golpe: esa es la señal. El cronómetro arranca en ese instante.
5. **RESULTADO.** Al pulsar, el display muestra el tiempo de reacción. Otra pulsación empieza de nuevo.

Y la trampa, que es lo que convierte el ejercicio en juego: **pulsar antes de que se apaguen es salida falsa**. El display muestra `SALIdA` y la partida se pierde. Como la espera es impredecible, no se puede anticipar: hay que reaccionar de verdad.

3.1 Lo que se ve en el display

SITUACIÓN	DISPLAY
En reposo	`----`
Justo al apagarse los LEDs	`AHOrA`
Resultado, por ejemplo 234 ms	`r0.234`
Salida falsa	`SALIdA`
Más de 2 s sin reaccionar	`LEnto`

> **Un display de siete segmentos no puede con todas las letras**, así que elige los textos entre los que la biblioteca sabe pintar: el espacio, `-`, los diez dígitos, `ABCEFHIJLOPSU`, `_` y `abcdhinortuy`. No existen la `G`, la `K`, la `M`, la `Q`, la `R`, la `V`, la `W`, la `X`, la `Z`, ni las mayúsculas `D`, `N`, `T` e `Y`. Si pides un carácter que no está, `tm1638_DisplayTxt` no falla en silencio: escribe `Error` seguido de la posición. Por eso el mensaje de salida es `AHOrA` y no `Go`.

4. El problema interesante: ¿de dónde sale el azar?

Si la espera del estado ARMADO fuese siempre la misma, el juego no valdría nada: a la tercera partida cualquiera la cuenta y se anticipa. Hace falta que sea impredecible.

Y ahí aparece un problema de verdad: **el STM32F401 no tiene generador de números aleatorios por hardware**. Algunos STM32 mayores sí lo llevan; este no. Un microcontrolador recién arrancado es la máquina más predecible del mundo: si el programa es el mismo y el estado inicial es el mismo, la secuencia es idéntica siempre.

> Prueba a poner una semilla fija y jugar tres partidas seguidas después de pulsar el botón de reinicio. Las esperas serán exactamente las mismas, en el mismo orden. Es un buen momento para entender qué significa «pseudoaleatorio».

Hay que buscar la entropía fuera del chip. Algunas fuentes posibles:

FUENTE	QUÉ TAL	POR QUÉ
Instante en que el jugador arranca	**buena**	Nadie pulsa dos veces en el mismo milisegundo. Es entropía humana, y no cuesta nada.
Bits bajos del ADC con la entrada al aire	razonable	Ruido térmico real, pero hay que configurar el ADC y el sesgo no es despreciable.
`HAL_GetTick()` en el arranque	mala	Vale casi lo mismo siempre: el programa tarda lo mismo en llegar hasta ahí.
Semilla fija	inútil	La misma partida una y otra vez.

La solución de referencia usa la primera: toma el valor de `HAL_GetTick()` en el momento exacto en que el jugador arma la salida y lo mezcla con la semilla. A partir de ahí, un generador congruencial lineal produce la secuencia.

GENERADOR CONGRUENCIAL LINEAL

```
semilla = semilla * 1664525u + 1013904223u;
```

Esas dos constantes no son arbitrarias: están elegidas para que el generador recorra los 2^{32} valores posibles antes de repetirse. Con otras cualesquiera el periodo puede ser ridículamente corto.

5. Medir bien el tiempo de reacción

Un tiempo de reacción humano ronda los **200 a 300 milisegundos**. Un buen piloto de Fórmula 1 baja de 200. Para que la medida signifique algo, el error del sistema tiene que ser muy inferior a eso.

El instante de la pulsación **se sella dentro de la función de atención a la interrupción**, no en el bucle principal:

CAPTURA POR INTERRUPCIÓN

```
void HAL_GPIO_EXTI_Callback(uint16_t pin)
{
    if (pin == B1_Pin) {
        tPulsacion = HAL_GetTick();   /* se sella aqui, no en el bucle
*/
        hayPulsacion = true;
    }
}
```

> Si la marca de tiempo se tomase en el bucle principal, el resultado dependería de lo que el bucle estuviera haciendo en ese momento: escribir en el display, por ejemplo, tarda milisegundos. La interrupción entra al instante, sea cual sea el punto del programa. **Esto es lo que P4 servía para poder hacer.**

En la memoria tendrás que estimar cuánto error introduce cada eslabón de la cadena: el rebote del pulsador, la latencia de la interrupción y la resolución de 1 ms de `HAL_GetTick()`.

6. Desarrollo paso a paso

Paso 1. La secuencia de luces

Sin juego todavía: al pulsar, que se enciendan los ocho LEDs uno a uno cada 400 ms y se queden encendidos. Cadencia con `HAL_GetTick()`, como en P3, sin `HAL_Delay`.

Paso 2. El apagón y la medida

Añade una espera fija de 2 s con todo encendido, luego apaga los LEDs y arranca la medida. Muestra el tiempo al pulsar. Con espera fija podrás comprobar que la medida es correcta antes de meter el azar.

Paso 3. La espera impredecible

Sustituye los 2 s fijos por un valor aleatorio entre 1 y 4 s, con la semilla tomada del instante en que el jugador arranca la partida. Comprueba que dos partidas seguidas dan esperas distintas.

Paso 4. La salida falsa

Detecta la pulsación durante SEMÁFORO y durante ARMADO, y trátala como salida falsa. Cuidado con un detalle: al llegar a MIDIENDO hay que **descartar cualquier pulsación pendiente**, o un rebote de la anterior contará como reacción de 0 ms.

Paso 5. El récord y el torneo

Guarda el mejor tiempo de la sesión. Y entonces la parte divertida: comparad los resultados de la clase.

Paso 6. Comprueba dónde se sella el instante

Todo el sentido de esta práctica está en que el tiempo se capture dentro de la interrupción y no en el bucle. Es una afirmación comprobable.

6. Pon un punto de ruptura en la línea de la rutina de atención que guarda el instante.
7. Juega una partida. Al pulsar, el programa se detendrá ahí.
8. Inspecciona el valor que acaba de guardarse y compáralo con el instante en que se apagaron los LEDs.

> **Y ahora fíjate en lo que le pasa a la medida.** El tiempo que el juego muestre al continuar será absurdo, porque has estado parado varios segundos en medio de la medición. No es un fallo: es el ejemplo más claro de que hay cosas que no se pueden medir mientras se observan. Explícalo en la memoria en dos líneas.

Para comprobar la afirmación de verdad —que capturar en la interrupción da mejores tiempos que capturar en el bucle— no sirve el depurador: hay que medir a velocidad real, que es lo que se hace en el torneo del paso anterior.

7. Criterios de aceptación

Nº	PRUEBA	RESULTADO EXIGIDO
CA-1	Arrancar una partida y observar la secuencia.	Los ocho LEDs se encienden uno a uno, y la secuencia completa dura **3,2 s ± 0,2 s**.
CA-2	Jugar cinco partidas seguidas y anotar la espera con todo encendido.	Las cinco esperas son **distintas** y todas caen entre 1 y 4 segundos.
CA-3	Reaccionar normalmente varias veces.	El tiempo mostrado está en el rango humano razonable, **entre 150 y 400 ms**. Nunca 0 ni valores absurdos.

Nº	PRUEBA	RESULTADO EXIGIDO
CA-4	Pulsar durante la secuencia de luces, y en otra partida con las ocho encendidas.	En los dos casos muestra `SALIdA` y no da ningún tiempo.
CA-5	Tras una salida falsa, jugar otra partida normal.	Funciona con normalidad: la salida falsa no deja el sistema en mal estado.
CA-6	Revisar el código.	El instante de la pulsación se toma **en la interrupción**, y el bucle principal no contiene esperas bloqueantes.

8. Criterios de evaluación

> **Condición previa.** El proyecto debe compilar **sin errores y sin avisos**. Un proyecto que no compila no se evalúa: se devuelve para corrección y se califica sobre 5.

CRITERIO	QUÉ SE VALORA	ALCANZADO A MEDIAS	PESO
1. Secuencia y temporización	CA-1: los ocho LEDs con la cadencia correcta.	La secuencia funciona pero con `HAL_Delay` en lugar de cadencia no bloqueante.	2,0
2. Espera impredecible	CA-2: esperas distintas en el rango pedido.	Es aleatoria pero con semilla fija: la misma serie tras cada reinicio.	2,0
3. Medida fiable	CA-3 y CA-6: captura en la interrupción, tiempos coherentes.	Mide bien pero sellando el instante en el bucle principal.	2,5

CRITERIO	QUÉ SE VALORA	ALCANZADO A MEDIAS	PESO
4. Salida falsa	CA-4 y CA-5: se detecta en las dos fases y el sistema se recupera.	La detecta en una sola de las dos fases, o queda colgado después.	1,5
5. Calidad del código	Máquina de estados legible, constantes con nombre, `volatile` donde toca.	Funciona pero mezcla decisiones y acciones, o faltan constantes.	1,0
6. Memoria	Las dos cuestiones del apartado 9, razonadas.	Responde una de las dos, o sin justificar.	1,0

9. Entregables

- **Proyecto de STM32CubeIDE** comprimido, **sin Debug ni Release**. Nombre: ACII_P7_Apellido1_Apellido2_Nombre.zip
- **Tabla con diez tiempos** de reacción propios, su media y el mejor.
- **Memoria breve**, máximo dos páginas, con estas dos cuestiones:
  - Estima el error total de tu medida: rebote del pulsador, latencia de la interrupción y resolución de `HAL_GetTick()`. ¿Cuál domina, y es despreciable frente a los 200 ms que tarda una persona?
  - Explica por qué tomar la semilla del instante en que el jugador arranca la partida da mejores resultados que tomarla de `HAL_GetTick()` nada más encender la placa.

10. Uso de asistentes de inteligencia artificial

Se permite el uso de asistentes de IA como herramienta de consulta y de depuración, con tres condiciones:

- **Declararlo** en la memoria: qué asistente y para qué.
- **Entender lo que entregas.** En la defensa se te puede pedir que expliques o modifiques cualquier línea.
- **No sustituir el trabajo de medida.** Los diez tiempos los consigues tú, jugando.

Anexo A. Ampliaciones, para quien acabe pronto

- **Media de las últimas cinco partidas**, además del mejor tiempo. Obliga a llevar un buffer circular.
- **Modo a dos jugadores**: uno usa el pulsador azul y otro una tecla del módulo. Gana quien reaccione antes, y el display dice cuál.
- **Resolución por debajo del milisegundo**, leyendo el registro de cuenta de SysTick además de `HAL_GetTick()`. Con 84 MHz la resolución teórica baja a 12 ns; discute si eso significa algo cuando lo que se mide es una persona.
- **Dificultad ajustable con el potenciómetro** de P5: que gobierne la velocidad de la secuencia o el rango de la espera.
- **Detección de trampa**: si alguien mantiene el pulsador apretado desde antes, el sistema debe darse cuenta.

Anexo B. Conexionado

El mismo de todas las prácticas. No hace falta cablear nada nuevo.

SEÑAL	PIN	PAPEL EN EL JUEGO
TM1638 · STB	PA10	Display y LEDs
TM1638 · CLK	PB4	
TM1638 · DIO	PB5	
Pulsador azul de usuario	PC13	El botón del juego, por interrupción externa